

PriceScope

Distributed Price Comparison Platform

Microservices + Celery + Redis for Distributed Product Comparison

Ayan Kumar Batabyal
Ashish Kumar Samantaray
Sayan Goswami

Department of Computer Science

April 28, 2026

Project Overview

- PriceScope is a distributed computing based product price comparison platform.
- It searches products across multiple e-commerce platforms simultaneously.
- Supported platforms: Amazon, eBay, Google Shopping, Walmart, and Target.
- Uses FastAPI microservices, Celery workers, and Redis message broker.
- Provides best price, best rating, and best value recommendations.

Problem Statement

- Users manually compare prices across multiple shopping websites.
- This process is time-consuming and inefficient.
- Different platforms show different prices, ratings, and offers.
- Need for a scalable distributed system to automate comparison.

Objectives

- Build a distributed product comparison platform.
- Enable parallel search across multiple services.
- Ensure fault tolerance and graceful degradation.
- Improve response speed using asynchronous processing.
- Provide intelligent ranking and analysis of products.

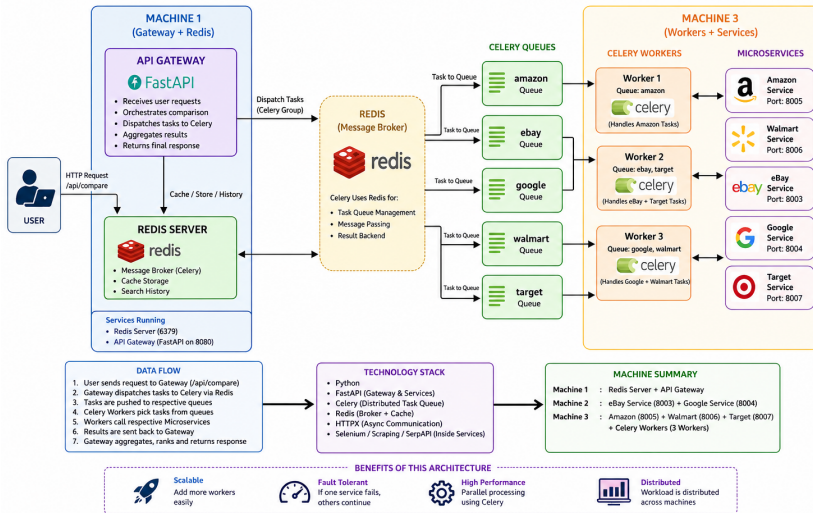
Technology Stack

- Python
- FastAPI (API Gateway + Microservices)
- Celery (Distributed Task Queue)
- Redis (Message Broker + Cache)
- Selenium + BeautifulSoup + HTTPX
- SerpAPI Integration
- HTML + CSS + JavaScript Frontend
- MCP Server for LLM integration

System Architecture

PriceScope – Distributed Architecture (3 Machine Setup)

Microservices + Celery + Redis for Distributed Product Comparison



Machine 1: Gateway + Redis

- API Gateway built using FastAPI.
- Receives user requests from frontend.
- Dispatches tasks to Celery queues.
- Aggregates results from all services.
- Redis used for:
 - Celery broker
 - Result backend
 - Cache storage
 - Search history

Machine 2 and 3: Workers + Services

- Dedicated workers handle assigned queues.
- Each worker calls specific microservices.
- Microservices run independently on separate ports.
- Enables true multiprocessing and scalability.

Service	Port	Method
Amazon	8005	Selenium
Google	8004	SerpAPI
Walmart	8006	Scraping
Target	8007	Scraping
ebay	8003	Scraping

Distributed Computing Features

- Parallel processing using Celery group() dispatch
- Dedicated queues per source
- Multiple worker processes
- Fault tolerance using Circuit Breaker Pattern
- Graceful degradation when services fail
- Automatic retries with exponential backoff
- Async fallback mode if Redis is unavailable

Celery Workflow

- 1 User sends request to `/api/compare`
- 2 Gateway creates Celery group tasks
- 3 Tasks are pushed to Redis queues
- 4 Workers consume tasks in parallel
- 5 Workers call respective microservices
- 6 Results are returned to Gateway
- 7 Gateway ranks and returns final response

Circuit Breaker Pattern

- Prevents repeated requests to failing services.
- Three states:
 - CLOSED: Normal operation
 - OPEN: Requests blocked temporarily
 - HALF_OPEN: Recovery testing state
- Improves fault tolerance and system stability.

API Endpoints

Endpoint	Method	Purpose
/api/compare	POST	Product comparison
/api/health	GET	Health check
/api/history	GET	Search history
/api/config	GET	System configuration

Frontend Features

- Modern responsive UI
- Real-time loading indicators
- Product ranking cards
- Best deal recommendations
- Search history tracking
- Service health monitoring
- Processing mode display (Celery / Async)

Result Ranking Logic

- Normalize prices to INR
- Remove duplicate products
- Calculate value score using:

$$\text{Value Score} = f(\text{Price}, \text{Rating}, \text{Reviews})$$

- Best Price = Minimum price
- Best Rating = Maximum rating
- Best Value = Highest value score

Real-World Use Cases

- **Smart Consumer Shopping:** Everyday users finding the best deals during high-traffic events like Black Friday or holiday sales.
- **Competitor Price Monitoring (B2B):** Retailers tracking rival pricing strategies in real-time to dynamically adjust their own inventory prices.
- **Retail Arbitrage & Dropshipping:** Independent sellers identifying price discrepancies across platforms to buy low and sell high.
- **Market & Inflation Research:** Data analysts tracking pricing trends, product availability, and inflation across consumer goods over time.

Testing Results

- eBay scraper successfully returned products
- Google Shopping returned multiple results
- Walmart scraper returned valid product data
- Amazon service works through Selenium
- Distributed mode successfully handled parallel execution

Performance Benefits

- Reduced search latency using parallel workers
- Improved scalability with distributed queues
- High availability with fallback strategies
- Better user experience through caching
- Improved reliability using retry + fault tolerance

Challenges Faced

- Managing multiple services across machines
- Handling scraper failures and anti-bot restrictions
- Redis and Celery worker synchronization
- Cross-platform deployment issues
- Debugging distributed failures

Future Improvements

- Add more e-commerce platforms
- Improve recommendation engine using LLMs
- Kubernetes deployment for cloud scaling
- Advanced analytics dashboard
- User authentication and saved preferences

Conclusion

- PriceScope demonstrates practical distributed computing.
- Combines microservices, Celery, and Redis effectively.
- Provides scalable, fault-tolerant product comparison.
- Shows real-world use of asynchronous distributed systems.

Thank You

Thank You